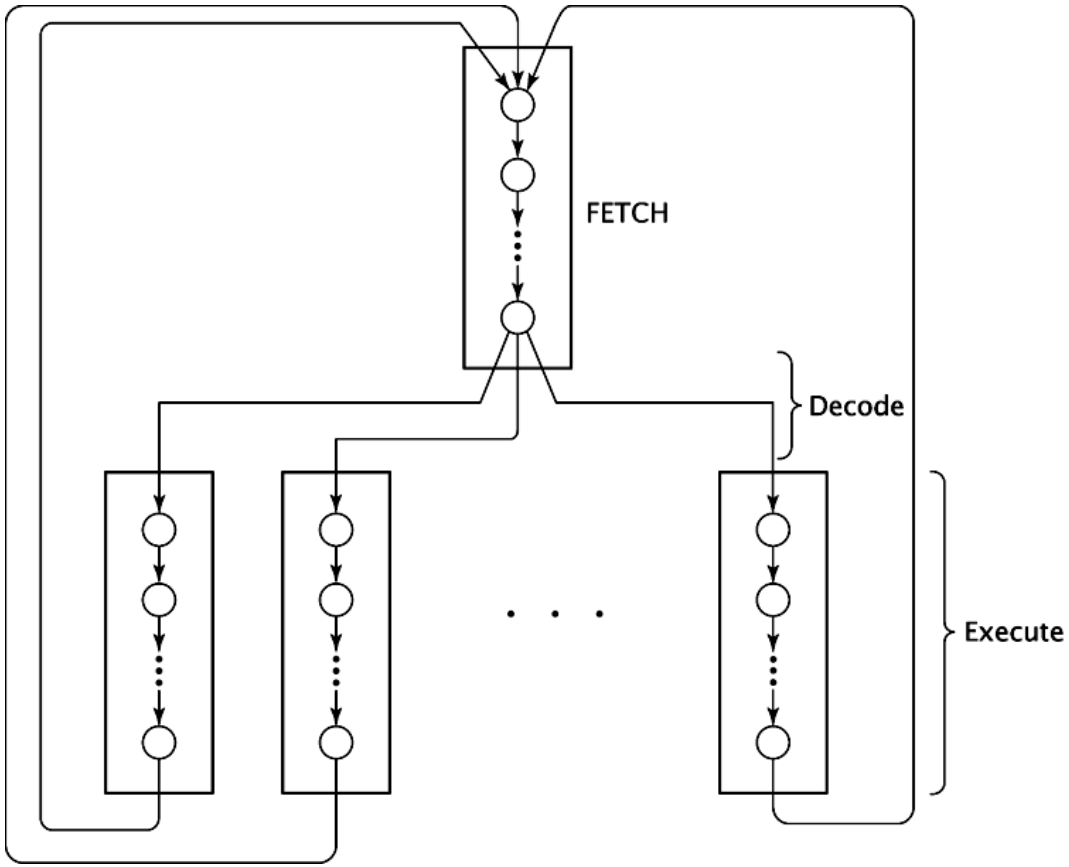
Generic CPU State Diagram

- Fetch cycle: Fetch an instruction from memory, then go to the decode cycle.
- Decode cycle: Decode the instruction – that is, determine which instruction has been fetched – then go to the execute cycle for that instruction.
- Execute cycle; Execute the instruction, then go to the fetch cycle and fetch the next instruction.



Very Simple CPU

- Access 64 bytes of memory
- Each byte has 8 bits
- Six bit address onoutput pins A[5..0]
- Reads 8 bits from memory on inputs D[7..0]
- CPU has on one programmer-accessable register, an 8-bit accumulator, AC

Instruction	Instruction Code	Operation
ADD	00AAAAAA	AC ← AC + M[AAAAAA]
AND	01AAAAAA	AC ← AC ^ M[AAAAAA]
JMP	10AAAAAA	GOTO AAAAAA
INC	11XXXXXX	AC ← AC + 1

Very Simple CPU – Additional Registers

- Six bit address register, AR
- Six bit program counter, PC
- Eight bit data register, DR
- Two bit instruction register, IR

Very Simple CPU – Fetching Instructions from Memory

- Send the address to memory by placing it on the address pins A[5..0]
- After allowing memory enough time to perform its internal decoding and to retriwe ve the desired instruction, send a signal to memory so that it outputs the instruction on its output pins.
- The address is stored in the program counter, PC

  FETCH1: AR ← PC
- CPU asserts a READ signal and increments PC

  FETCH2: DR ← M, PC ← PC + 1
- Copy two high order bits of DR to IR and six low-order bits of DR to AR

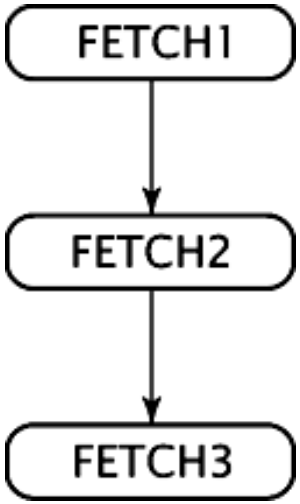
  FETCH3: IR ← DR[7..6], AR ← DR[5..0]

Fetch Cycle for Very Simple CPU

FETCH1: AR ← PC

FETCH2: DR ← M, PC ← PC + 1

FETCH3: IR ← DR[7..6], AR ← DR[5..0]



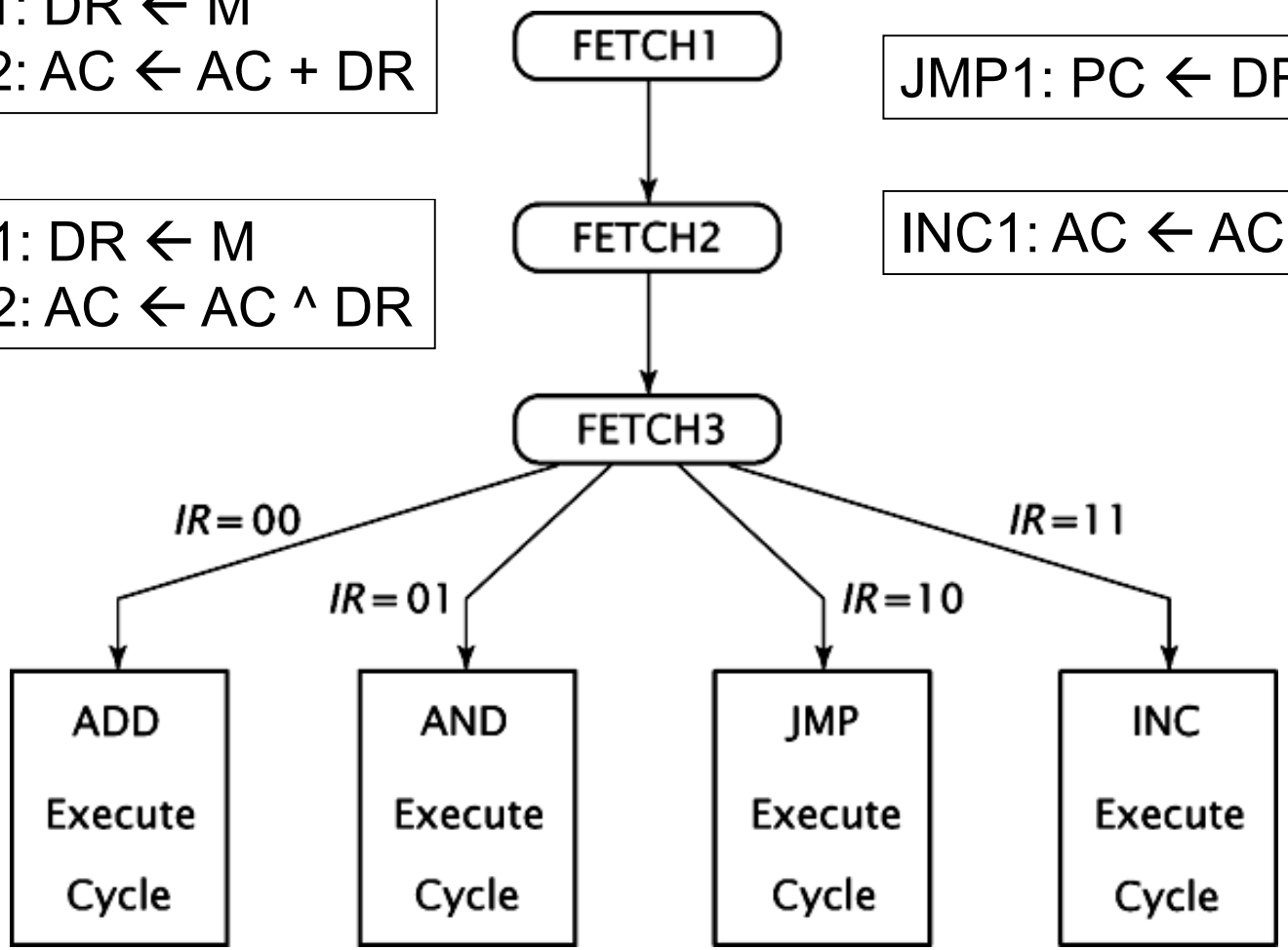
Fetch and Decode Cycles for Very Simple CPU

ADD1: DR ← M
ADD2: AC ← AC + DR

AND1: DR ← M
AND2: AC ← AC ^ DR

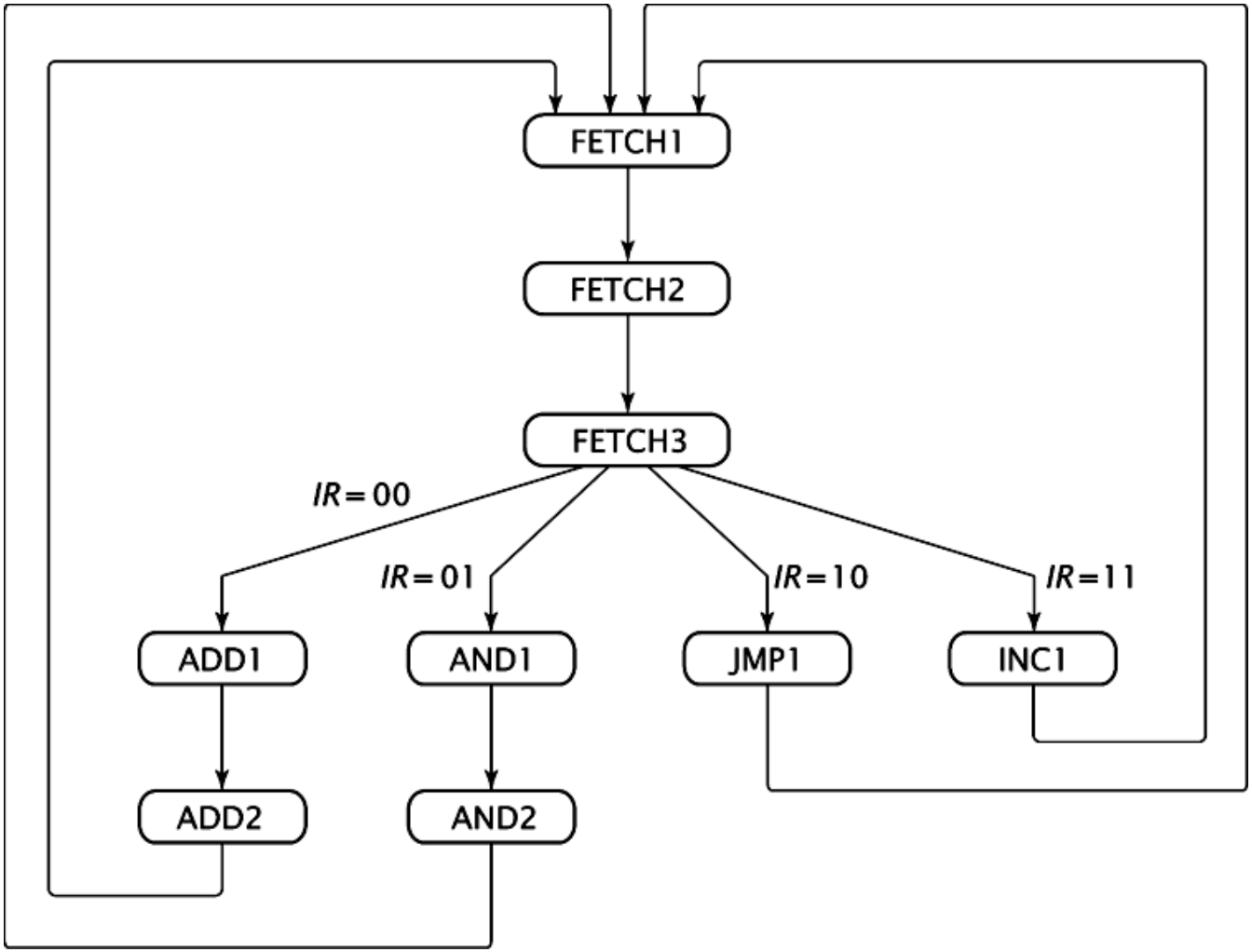
JMP1: PC ← DR[5..0]

INC1: AC ← AC + 1



Complete State Diagram for Very Simple CPU

This diagram includes the data paths for all the operations



Establishing Required Data Paths

- Operations associated with each state of the CPU are:

  FETCH1: AR ← PC

  FETCH2: DR ← M, PC ← PC + 1

  FETCH3: IR ← DR[7..6], AR ← DR[5..0]

  ADD1: DR ← M

  ADD2: AC ← AC + DR

  AND1: DR ← M

  AND2: AC ← AC ^ DR

  JMP1: PC ← DR[5..0]

  INC1: AC ← AC + 1

Establishing Required Data Paths

- Regroup the operations, without regard for the cycles in which they occur, by registers whose contents they modify.

  AR: AR ← PC; AR ← DR[5..0]

  PC: PC ← PC + 1; PC ← DR[5..0]

  DR: DR ← M

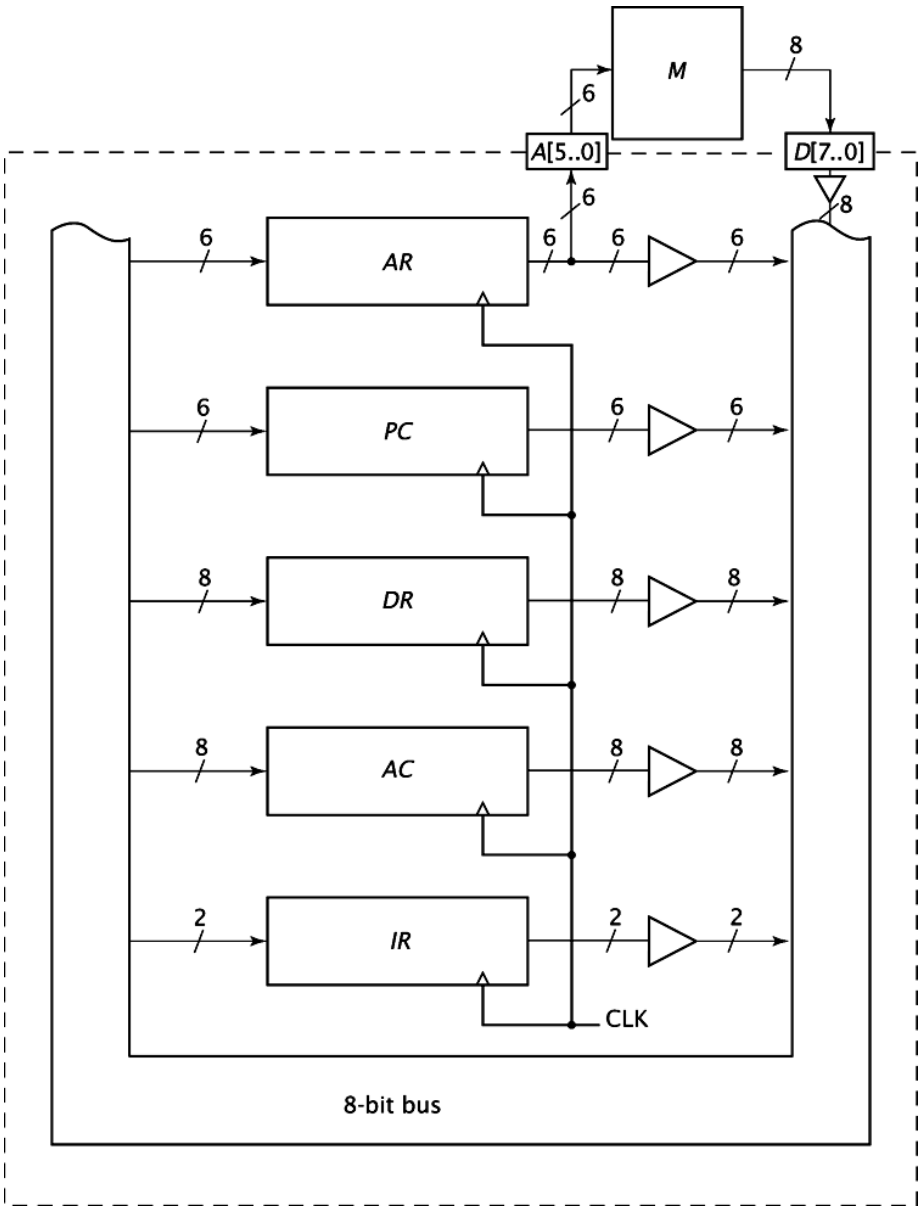
  IR: IR ← DR[7..6]

  AC: AC ← AC + DR; AC ← AC ^ DR; AC ← AC + 1
- Connect every component to the system bus

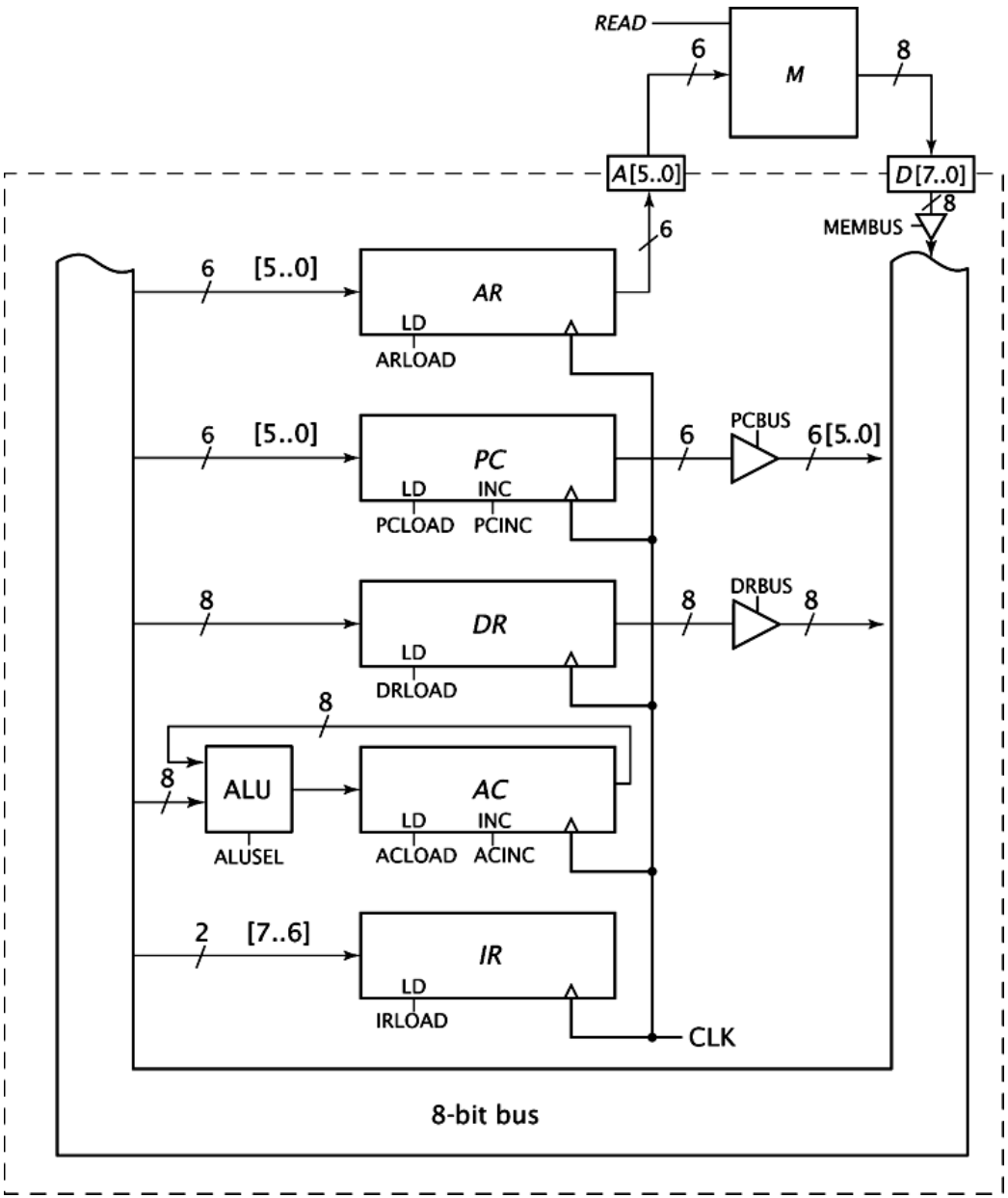
Preliminary Register Section for the Very Simple CPU

Simple Observations:

1. AR only supplies its data to memory – don't need to connect to any other component
2. IR does not supply data to any other component via the internal bus – can remove output connection
3. AC does not supply itws data to any component – remove connection to internal bus
4. The bus is 8 bits wide, but not all data transfers are 8 bits (some are 6, some are 2) – need to specify which registers send and receive to and from which bus
5. AC must be able to load the sum of AC and DR, logical AND of AC and DR. CPU needs to include an ALU to do that.

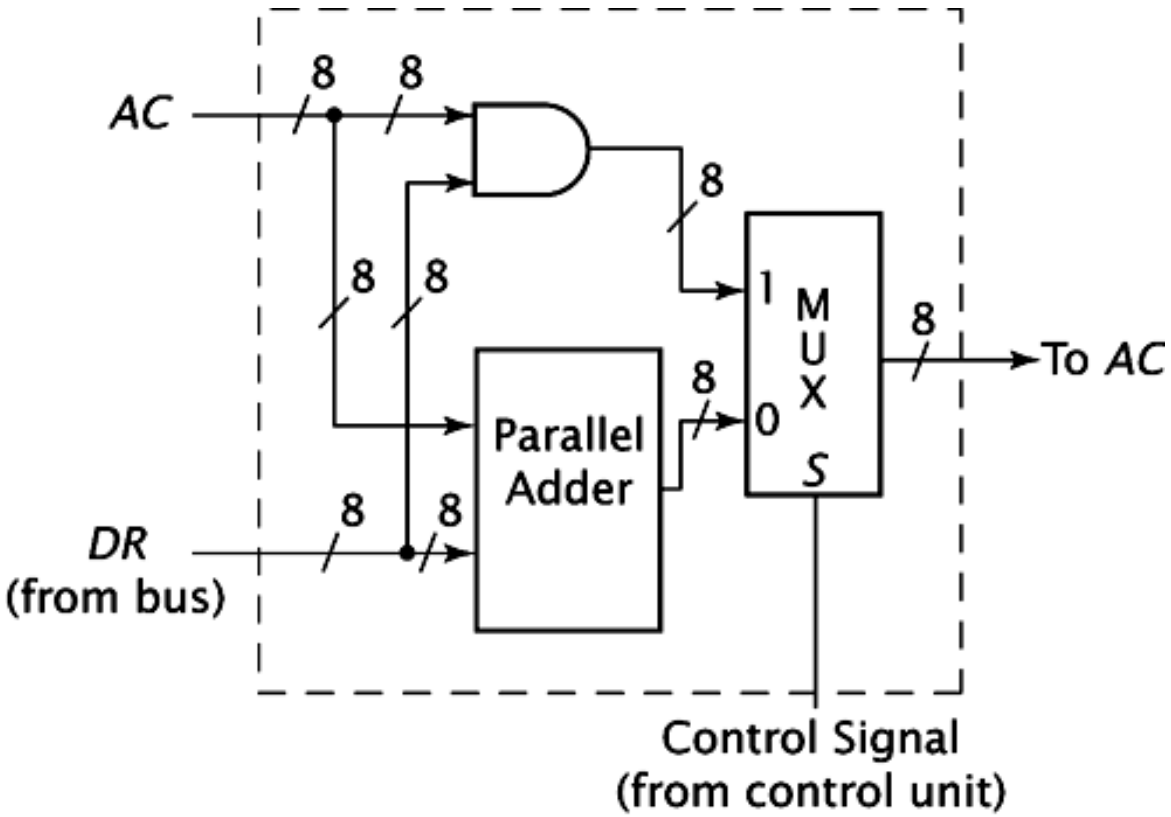


Final Register Section for the Very Simple CPU

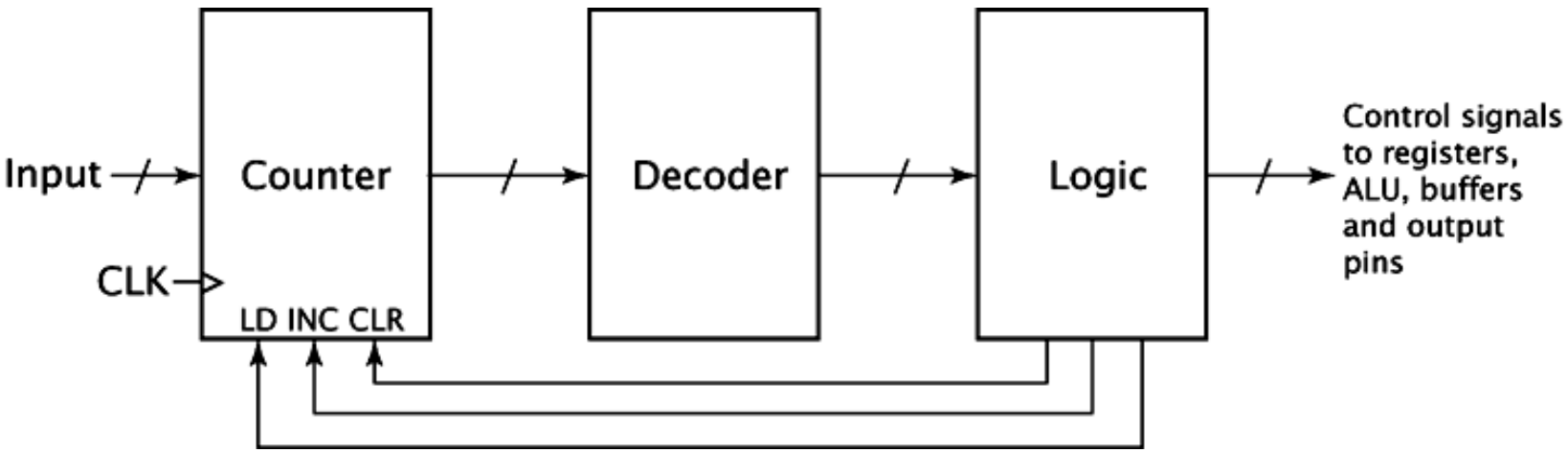


A Very Simple ALU

Create separate hardware for each function and use a multiplexer to select the function results



Generic Hardwired Control Unit



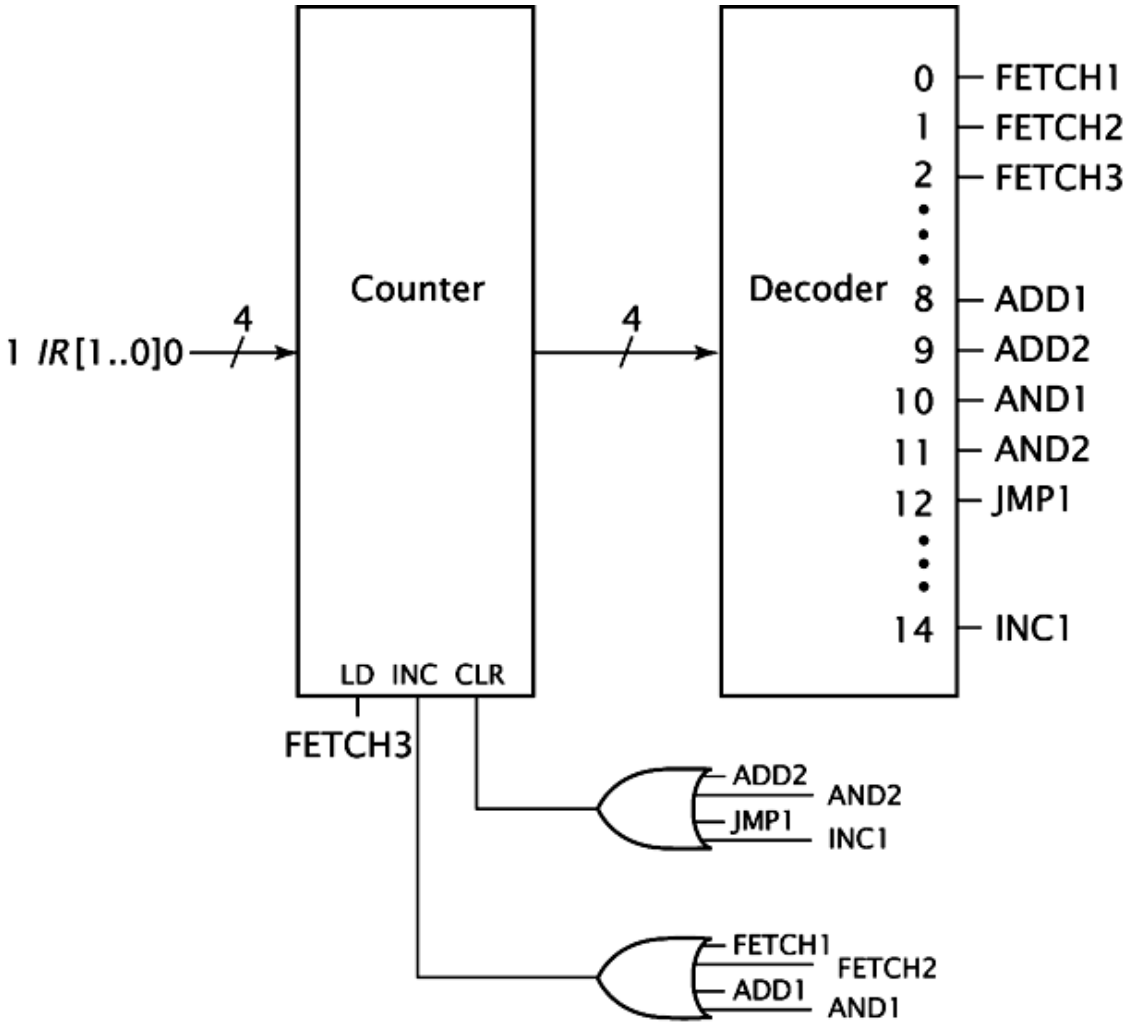
- Counter contains current state
- Generates individual signals from current state
- Logic to take individual state signals and generate control signals for each component, as well as the signals to control counter

Very Simple CPU has Nine States

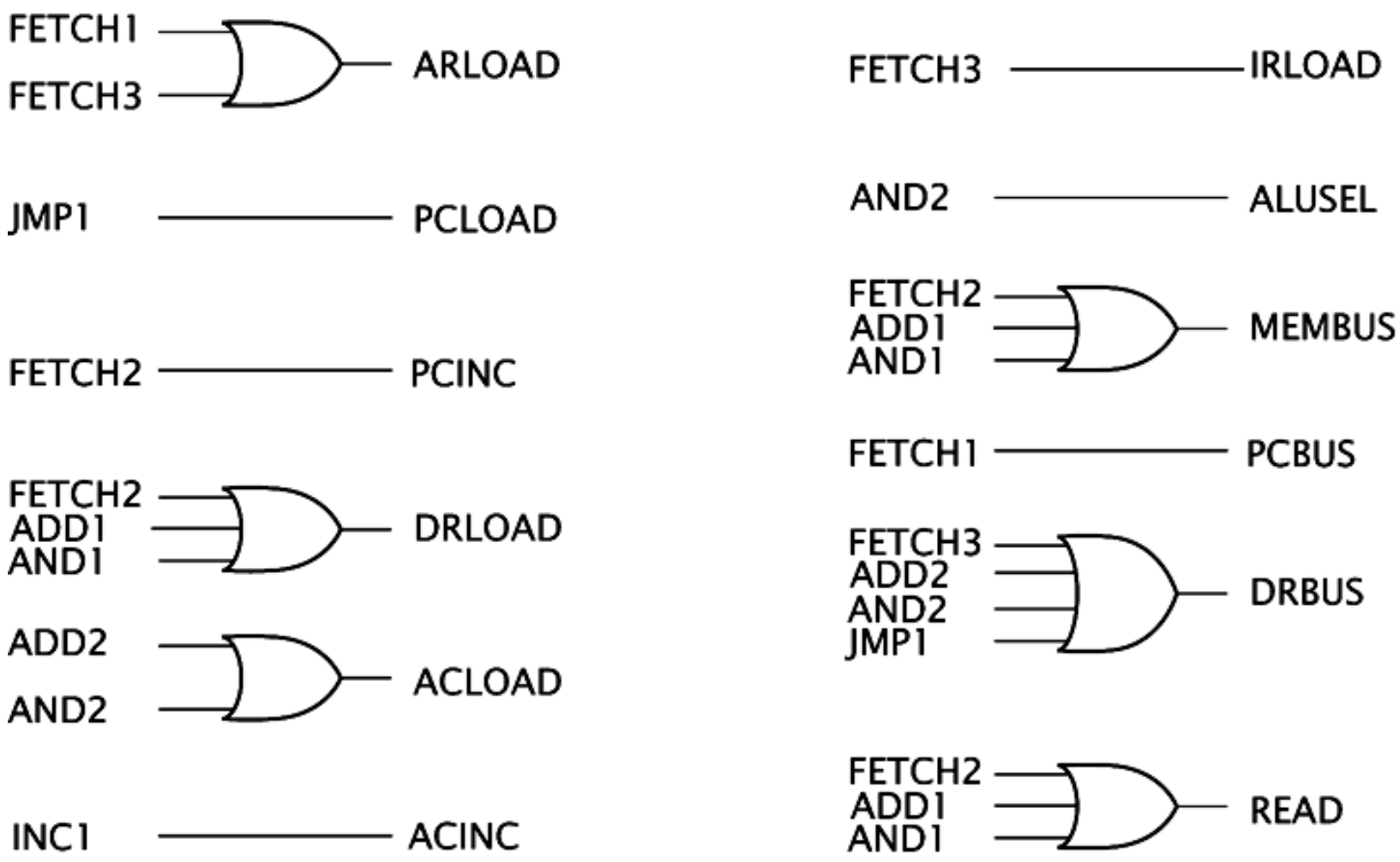
- Need four bit counter and a 4-to-16 bit decoder. Seven outputs of decoder will not be used

1. Assign FETCH1 to counter value 0 and use the CLR input of the counter to reach this state.
2. Assign sequential states to sequential counter values and use the INC input of the counter to traverse these states. CPU would assign FETCH2 to counter value 1 and FETCH3 to counter value 2, ADD1 and ADD2 to consecutive counter values as well as AND1 and AND2
3. Assign the first state fo each execute routine based on the instruction opcodes and the maximum number of states in the execute routines. Use the opcodes to generate the data input to the counter and the LD input to the counter to reach the proper execute routine.

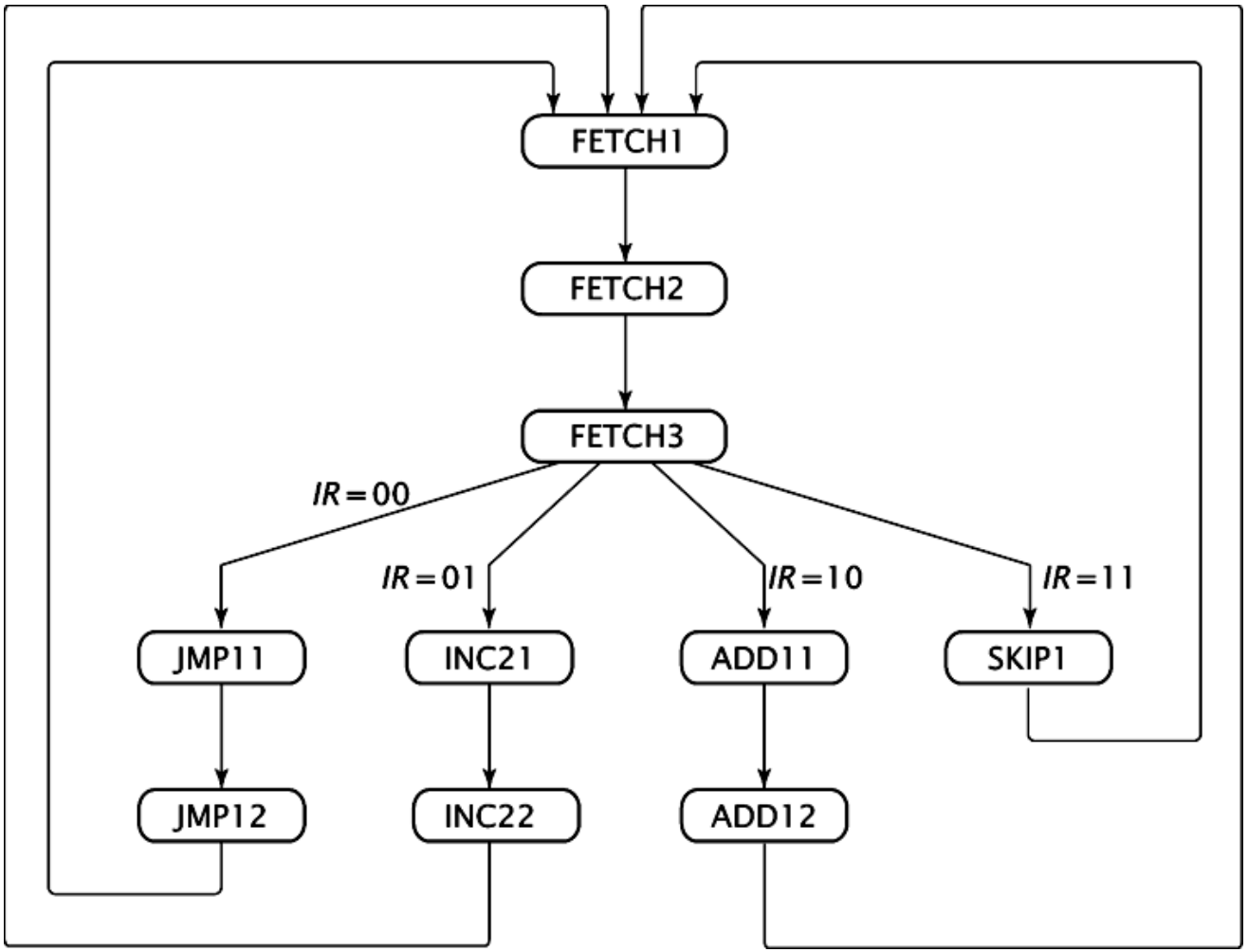
Hardwired Control Unit for Very Simple CPU



Control Signal Generation for Very Simple CPU

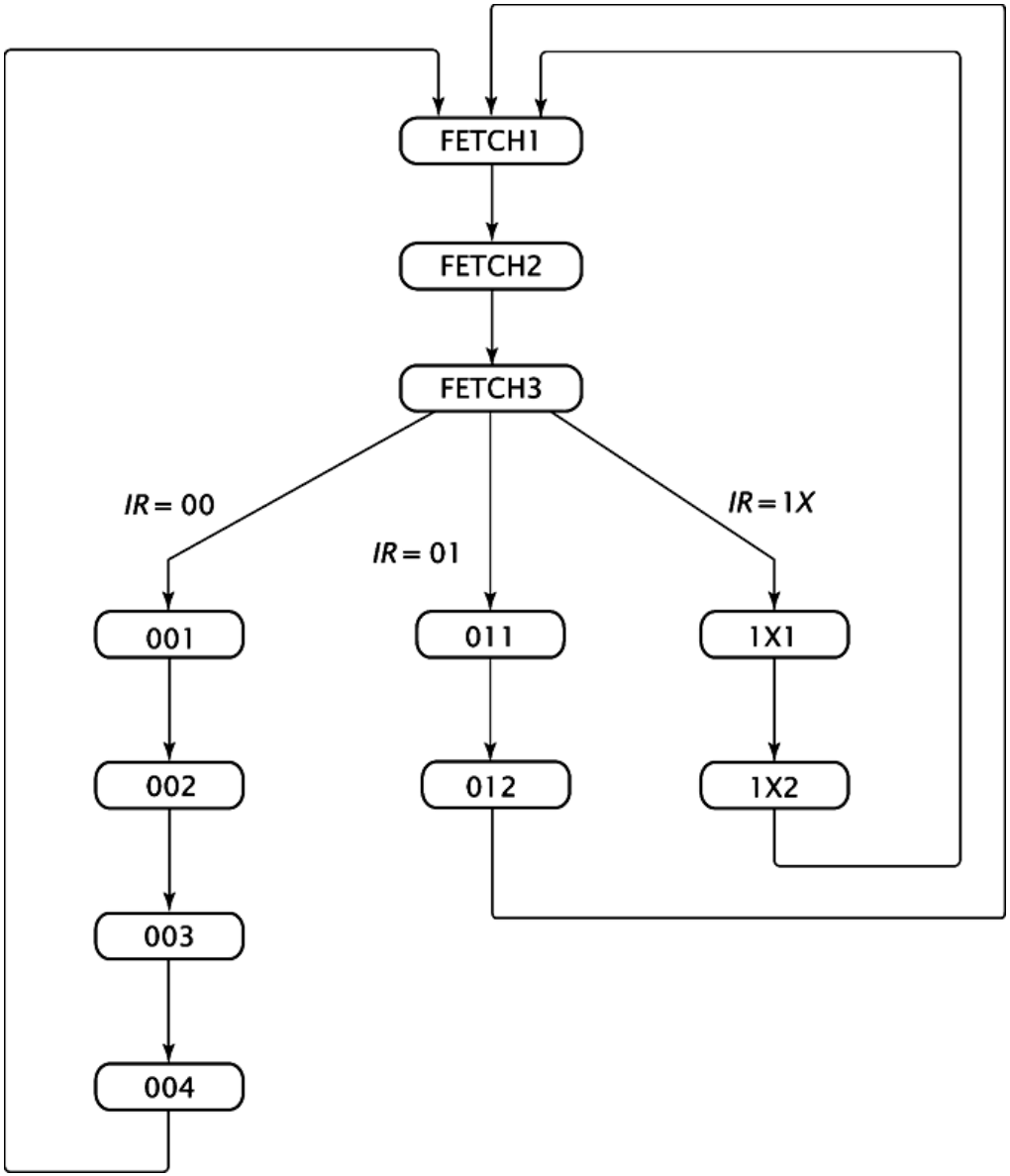


Page 260, Problem 6.1

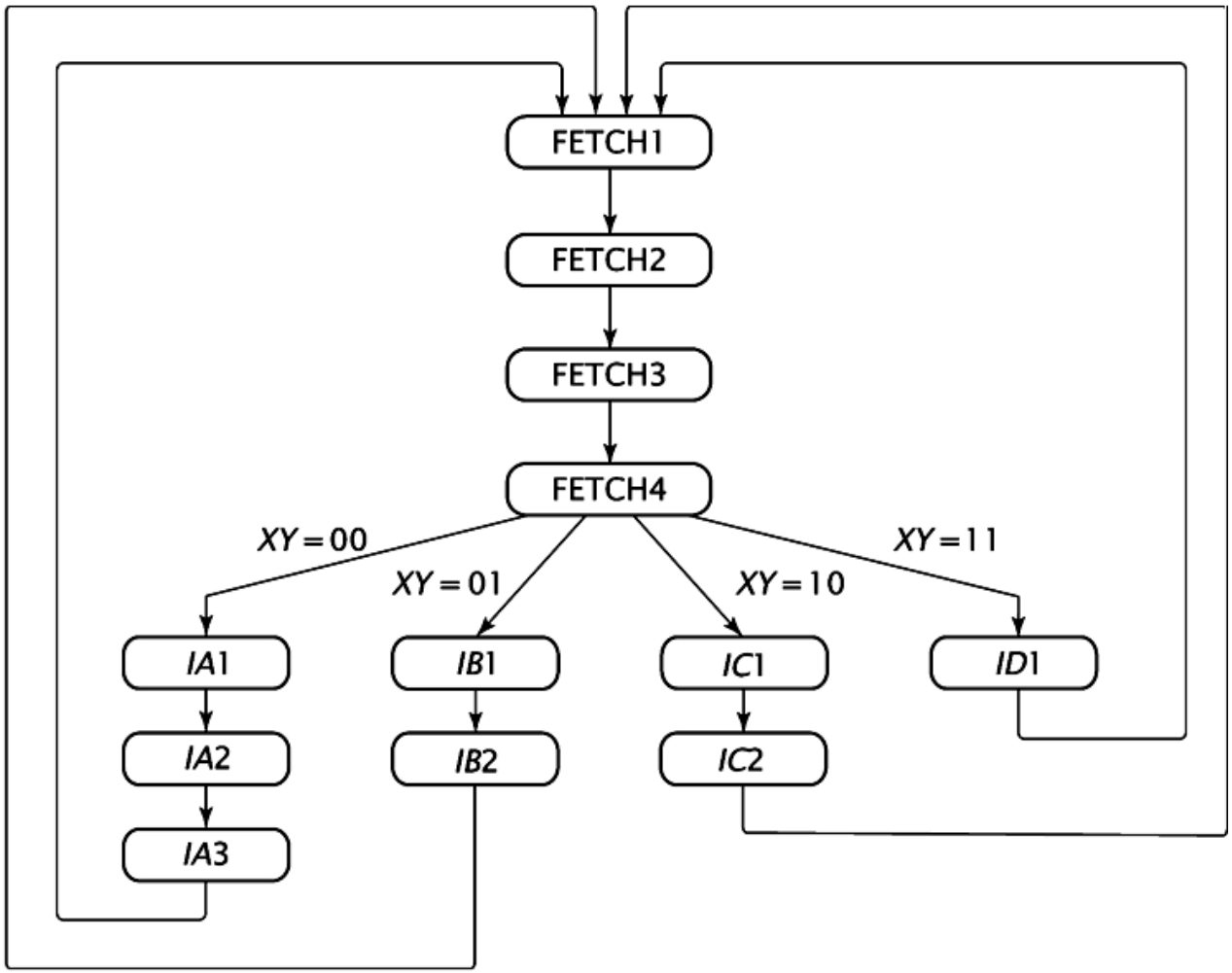


Page 261,

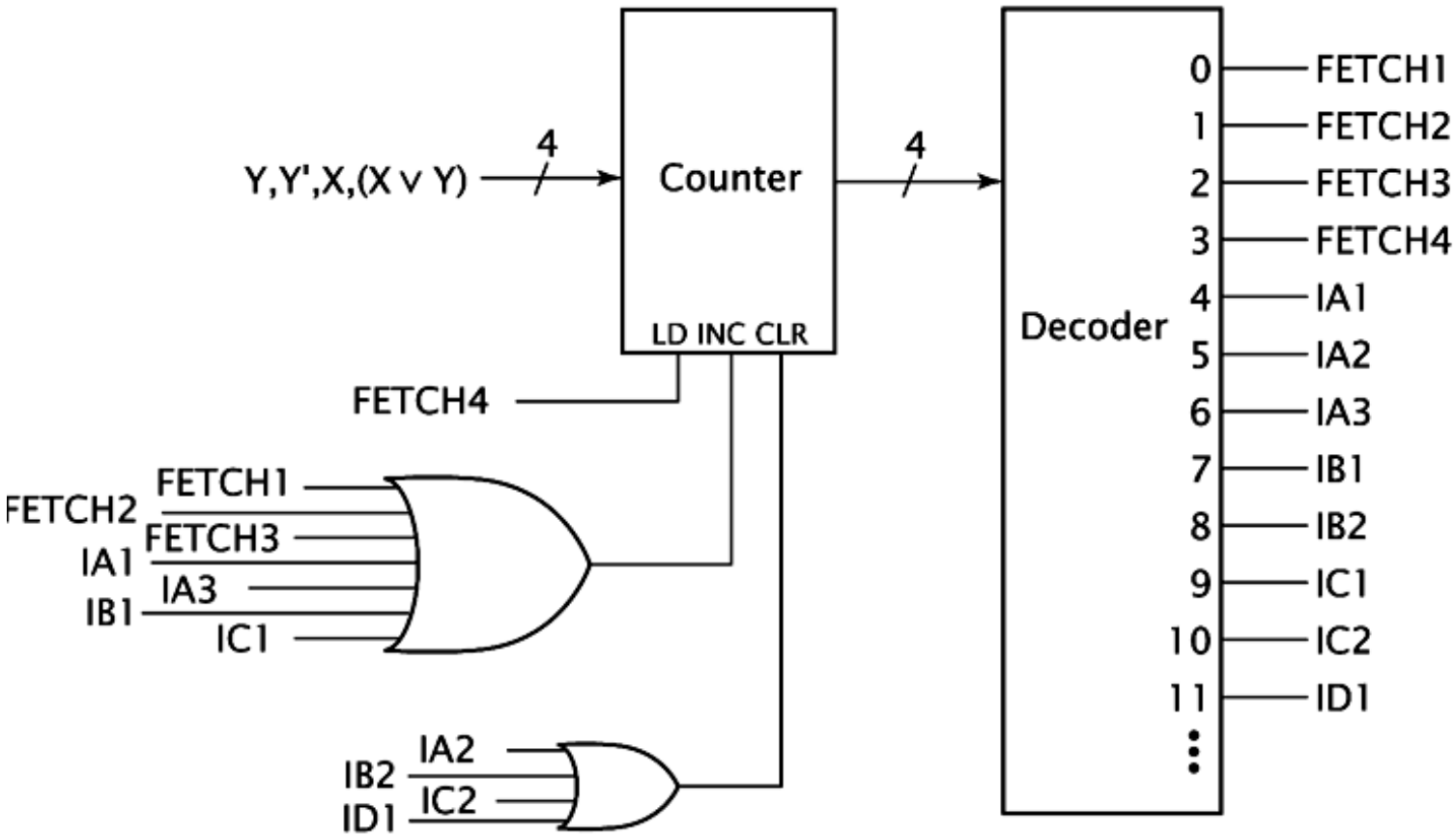
Problem 6.2



Page 262, Problem 6.4



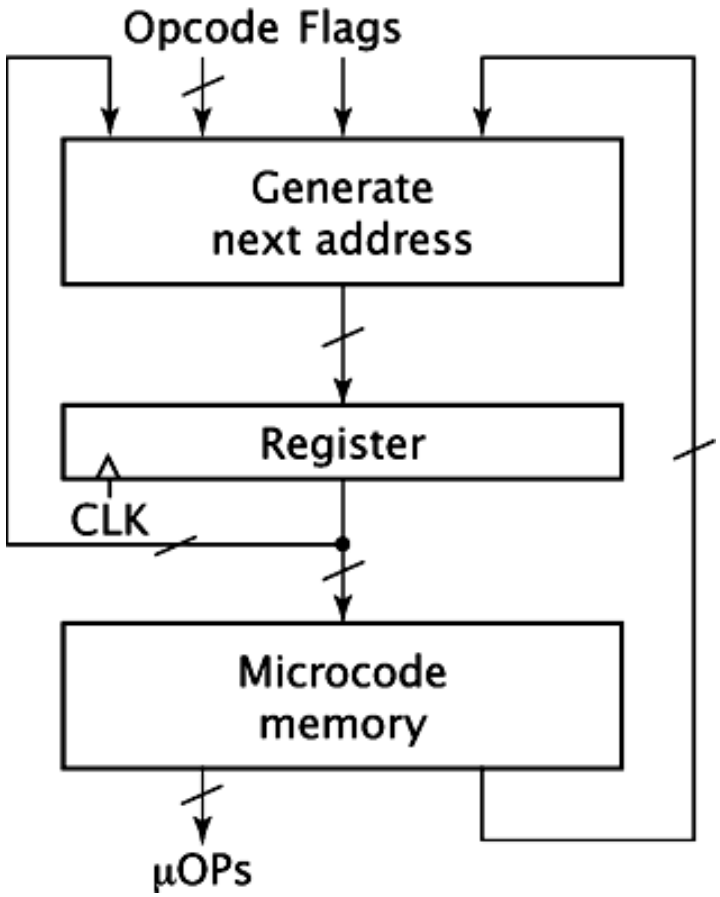
Page 262, Problem 6.4 Continued



Microsequencer Control Unit Design

Generic microsequencer organiation

- µOPs: input to combinatorial logic to generate the CPU's control signals, or they directly produce the control signals (in state diagram, outputs associated with states)
- Second output is used to generate the next address
    - Input, along with opcode and flag values are used to generate address of next microinstruction (equivalent of making transition from one state to the next)
- Generate next address: some possible addresses
    - Next address in microcode memory which is the current address + 1
    - Another possible address is the absolute address supplied by microcode memory



Generic microinstruction format

SELECT	ADDR	MICRO-OPERATIONS

- SELECT field: determines source of address of next microinstruction
- ADDR field: specifies an absolute address. Used when performing absolute jump
- MICRO-OPERATIONS field: three primary methods
    - Horizontal microcode
    - Vertical microcode
    - Direct generation of control signals

Horizontal Microcode

- List every micro operation performed by CPU
- Assign one bit in microcode for each micro operation
  - Can result in large microcode (for example, for a CPU with 50 microinstruction, that would be a microinstruction with 50 bits – pretty large
- Need a more efficient operation

Vertical Microcode

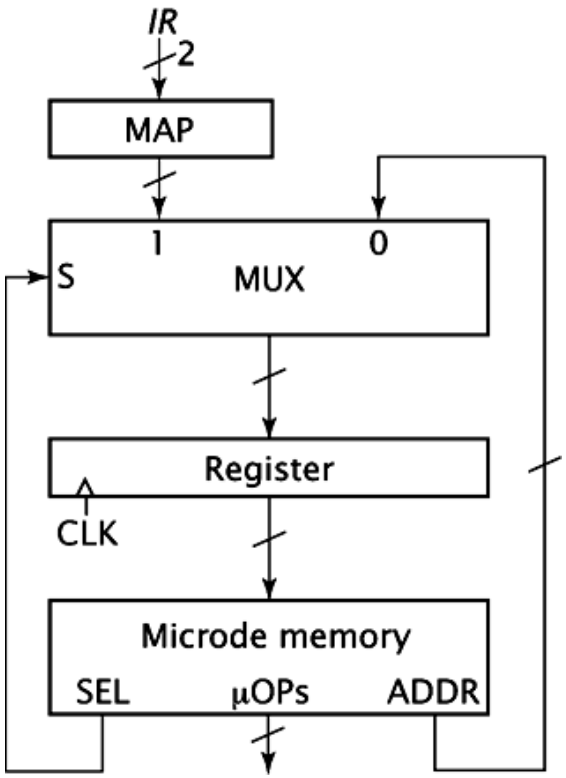
- Micro operations are grouped into fields
  - Each micro operation assigned a unique encoded value to this field
    - 16 micro operations could be encoded using four bits, with each micro operation assigned a unique binary field value from 0000 to 1111 (one would be need for a no op operation)
- Requires fewer bits for each micro operation but bust include a decoder.
- Both vertical and horizontal, CPU must convdert micro operation signal to the control signals that load, clear, and increment registers

Direct Generation of Control Signals

- Does away with micro operations
- Stores values of control signals directly in microinstruction
  - One bit connected directly to load signal, another to increment input of program counter, etc.
- Doesn’t require additional logic, but is less readable and more difficult to debug

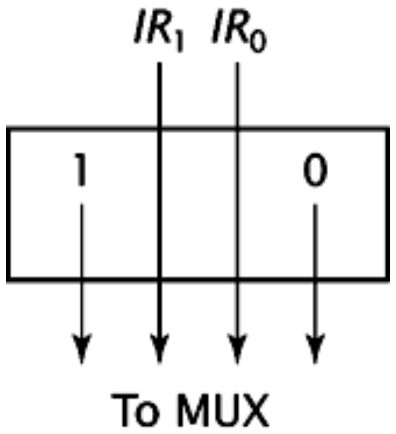
Microsequencer for Very Simple CPU

- Two possible next addresses
  - Opcode mapping
  - Absolute jump
- MUX: use to select the correct address
- Input to MUX, Register, and Microde memory and ADDR output from Microde memory is 4 bits – min bits need to determine one of 9 states of CPU



Mapping logic for the Very Simple Microsequencer

State	Address
FETCH1	0000 (0)
FETCH2	0001 (1)
FETCH3	0010 (2)
ADD1	1000 (8)
ADD2	1001 (9)
AND1	1010 (10)
AND2	1011 11)
JMP1	1100 (12)
INC1	1110 (14)



Assign each state of the finite state machine to an address in microcode

By choosing the addresses wisely we can minimize connections

Partial Microcode for the Very Simple Microsequencer

State	Address	SEL	ADDR
FETCH1	0000 (0)	0	0001
FETCH2	0001 (1)	0	0010
FETCH3	0010 (2)	1	xxxx
ADD1	1000 (8)	0	1001
ADD2	1001 (9)	0	0000
AND1	1010 (10)	0	1011
AND2	1011 11)	0	0000
JMP1	1100 (12)	0	0000
INC1	1110 (14)	0	0000

- SEL = 0 will get next address from ADDR field
- FETCH3 must map to correct execute routine, so SEL = 1 to use that address

Micro operations and their mnemonics for the Very Simple Microsequencer

- Need to list every micro operation so we can develop the code.
- The Very Simple CPU has nine micro operations.
- Need one bit for each operation.

Mnemonics	Micro Operation
ARPC	AR ← PC
ARDR	AR ← DR[5..0]
PCIN	PC ← PC + 1
PRDR	PC ← DR[5..0]
DRM	DR ← M
IRDR	IR ← DR[7..6]
PLUS	AC ← AC + DR
AND	AC ← AC ^ DR
ACIN	AC ← AC + 1

Establishing Required Data Paths

- Operations associated with each state of the CPU are:

  FETCH1: AR ← PC

  FETCH2: DR ← M, PC ← PC + 1

  FETCH3: IR ← DR[7..6], AR ← DR[5..0]

  ADD1: DR ← M

  ADD2: AC ← AC + DR

  AND1: DR ← M

  AND2: AC ← AC ^ DR

  JMP1: PC ← DR[5..0]

  INC1: AC ← AC + 1

Preliminary horizontal microcode for the Very Simple Microsequencer

State	Address	SEL	ARPC	ARDR	PCIN	PCDR	DRM	IRDR	PLUS	AND	ACIN	ADDR
FETCH1	0000 (0)	0	1	0	0	0	0	0	0	0	0	0001
FETCH2	0001 (1)	0	0	0	1	0	1	0	0	0	0	0010
FETCH3	0010 (2)	1	0	1	0	0	0	1	0	0	0	xxxx
ADD1	1000 (8)	0	0	0	0	0	1	0	0	0	0	1001
ADD2	1001 (9)	0	0	0	0	0	0	0	1	0	0	0000
AND1	1010 (10)	0	0	0	0	0	1	0	0	0	0	1011
AND2	1011 11)	0	0	0	0	0	0	0	0	1	0	0000
JMP1	1100 (12)	0	0	0	0	1	0	0	0	0	0	0000
INC1	1110 (14)	0	0	0	0	0	0	0	0	0	1	0000

Optomize

- For all states, ARDR and IRDR have the same value
- Don’t need two outputs
- Use one output to drive both micro operations, AIDR, combining
  - AR ← DR[5..0] and IR ← DR[7..6]

Optimized horizontal microcode for the Very Simple Microsequencer

State	Address	SEL	ARPC	AIDR	PCIN	PCDR	DRM	PLUS	AND	ACIN	ADDR
FETCH1	0000 (0)	0	1	0	0	0	0	0	0	0	0001
FETCH2	0001 (1)	0	0	0	1	0	1	0	0	0	0010
FETCH3	0010 (2)	1	0	1	0	0	0	0	0	0	xxxx
ADD1	1000 (8)	0	0	0	0	0	1	0	0	0	1001
ADD2	1001 (9)	0	0	0	0	0	0	1	0	0	0000
AND1	1010 (10)	0	0	0	0	0	1	0	0	0	1011
AND2	1011 11)	0	0	0	0	0	0	0	1	0	0000
JMP1	1100 (12)	0	0	0	0	1	0	0	0	0	0000
INC1	1110 (14)	0	0	0	0	0	0	0	0	1	0000

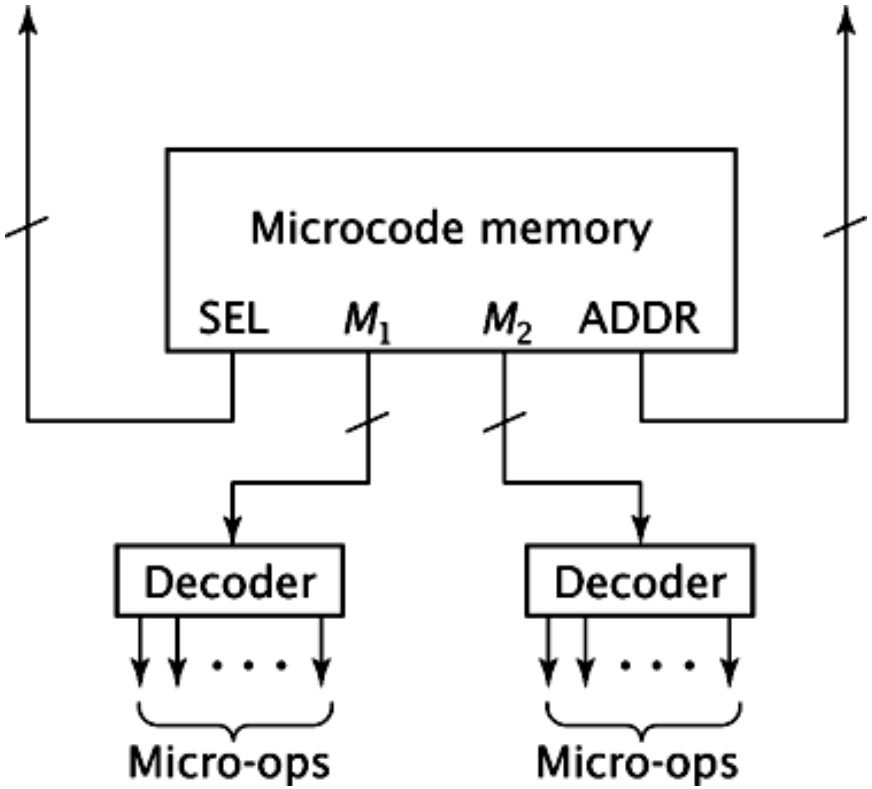
Control signal values for the Very Simple CPU

From the previous table the control signals can be generated

Signal	Value
ARLOAD	ARPC v AIDR
PCLOAD	PCDR
PCINC	PCIN
DRLOAD	DRM
ARLOAD	PLUS v AND
ARIINC	ACIN
IRLOAD	AIDR
ALUSELL	AND
MEMBUS	DRM
PCBUS	ARPC
DRBUS	AIDR v PCDR v PLUS v AND
READ	DRM

Generic generation fo micro operations from vertical microcode

- Vertical code table consists mostly of zeros – 85%
- Vertical micro operations are grouped into fields
- Can use decoders to generate the instructions



Guidelines to design microsequencer using vertical microcode

1. Whenever two micro operations occur during the same state, assign them to different fields.
2. Include a NOP in each field if necessary.
   - Needed because some value must be output every cycle
3. Distribute the remaining micro operations to make best use of the micro operation field bits.
4. Group together micro operations that modify the same registers in the same fields.

Find micro operations that can run concurrently

- DRM and PCIN both occur in FETCH2, must be assigned to different fields
  - Will need at least two fields for the Very Simple CPU

M1	M2
NOP	NOP
DRM	PCIN

- Since PCIN and PCDR both modify PC, add PCDR to M2
- Then arbitrarily assign the remaing micro operations to the fields, keeping micro operations that change the same register in the same field.

Micro operation field assignments and values

MI	
Value	Micro Operation
000	NOP
001	DRM
010	ARPC
011	AIDR
100	PCDR
101	PLUS
110	AND
111	ACIN

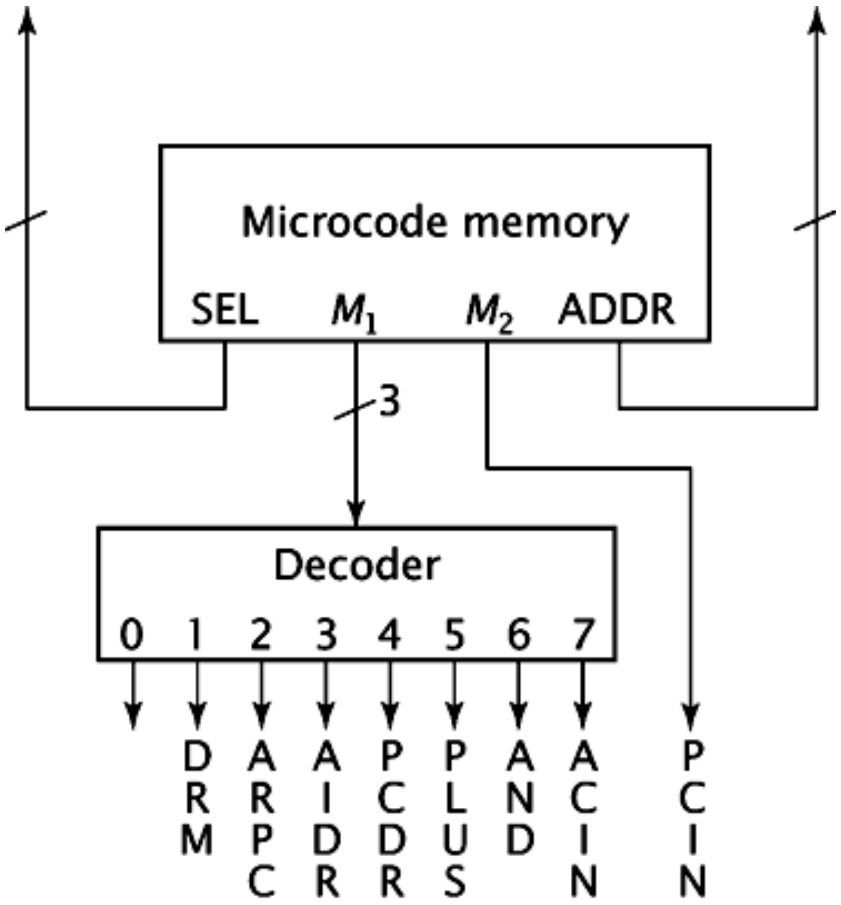
M2	
Value	Micro Operation
0	NOP
1	PCIN

- To optimize, it is best to maximize to a power of two, so is possible, put eight micro operations in M1 and two in M2.
- M1 requires 3 bits, M2 only one bit, for total of 4 bits

Vertical microcode for the Very Simple Microsequencer

State	Address	SEL	M1	M2	ADDR
FETCH1	0000 (0)	0	010	0	0001
FETCH2	0001 (1)	0	001	0	0010
FETCH3	0010 (2)	1	011	1	xxxx
ADD1	1000 (8)	0	001	0	1001
ADD2	1001 (9)	0	101	0	0000
AND1	1010 (10)	0	001	0	1011
AND2	1011 11)	0	110	0	0000
JMP1	1100 (12)	0	100	0	0000
INC1	1110 (14)	0	111	0	0000

Generating micro operations for vertical microcode for the Very Simple CPU



Directly Generating Control Signals from the Microcode

- Use one bit for each control signal
  - Set to 1, it is active
  - Set to 0, it is not active
- Example

  FETCH2: DR ← M and PC ← PC + 1

  READ output data from memory

  MEMBUS to allow data onto internal bus

  DRLOAD to load data from bus into DR

  PCINC to perform second micro operation

Set those four to 1, the others to 0

Microcode to directly generate control signals for the Very Simple Microsequencer

State	Address	SEL	ARLOAD	PCLOAD	PCINC	DRLOAD	ACLOAD	ACINC	IRLOAD	ALUSEL	MEMBUS	PCBUS	DRBUS	READ	ADDR
FETCH1	0000 (0)	0	1	0	0	0	0	0	0	0	0	1	0	0	0001
FETCH2	0001 (1)	0	0	0	1	1	0	0	0	0	1	0	0	1	0010
FETCH3	0010 (2)	1	1	0	0	0	0	0	1	0	0	0	1	0	xxxx
ADD1	1000 (8)	0	0	0	0	1	0	0	0	0	1	0	0	1	1001
ADD2	1001 (9)	0	0	0	0	0	1	0	0	0	0	0	1	0	0000
AND1	1010 (10)	0	0	0	0	1	0	0	0	0	1	0	0	1	1011
AND2	1011 11)	0	0	0	0	0	1	0	0	1	0	0	1	0	0000
JMP1	1100 (12)	0	0	1	0	0	0	0	0	0	0	0	1	0	0000
INC1	1110 (14)	0	0	0	0	0	0	1	0	0	0	0	0	0	0000

Optimized microcode to directly generate control signals

State	Address	SEL	ARLOAD	PCLOAD	PCINC	DRM	ACLOAD	ACINC	IRLOAD	ALUSEL	PCBUS	DRBUS	ADDR
FETCH1	0000 (0)	0	1	0	0	0	0	0	0	0	1	0	0001
FETCH2	0001 (1)	0	0	0	1	1	0	0	0	0	0	0	0010
FETCH3	0010 (2)	1	1	0	0	0	0	0	1	0	0	1	xxxx
ADD1	1000 (8)	0	0	0	0	1	0	0	0	0	0	0	1001
ADD2	1001 (9)	0	0	0	0	0	1	0	0	0	0	1	0000
AND1	1010 (10)	0	0	0	0	1	0	0	0	0	0	0	1011
AND2	1011 11)	0	0	0	0	0	1	0	0	1	0	1	0000
JMP1	1100 (12)	0	0	1	0	0	0	0	0	0	0	1	0000
INC1	1110 (14)	0	0	0	0	0	0	1	0	0	0	0	0000

Advantage to Directly Generating Control Signals

- Does not require additional logic to convert the outputs of the microcode memory to control signals
- However, less readable and more difficult to debug